Báo cáo phân tích hiệu năng thuật toán sắp xếp

Assignment 1: Selection Sort, Insertion Sort và Quick Sort (C/C++)

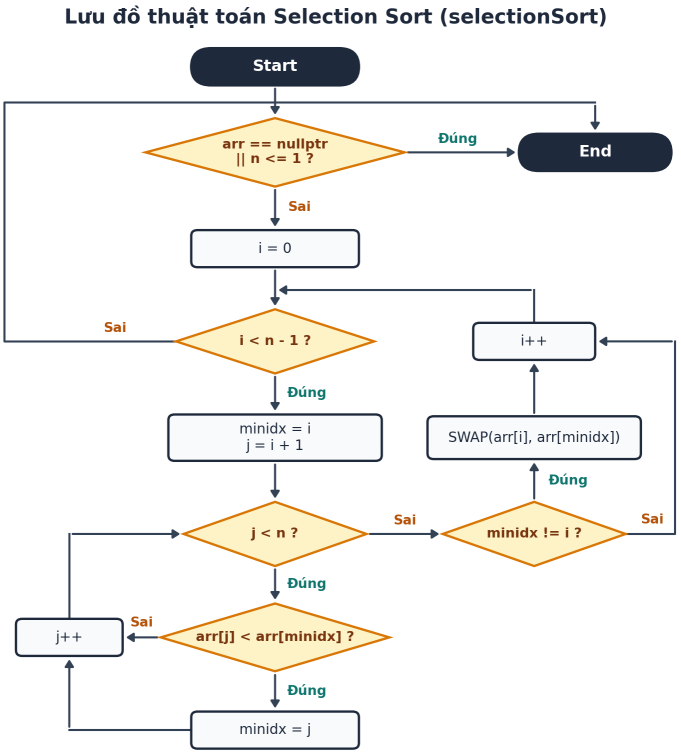
Sinh viên thực hiện: Lương Quang Huy | **MSSV:** 202618357

Báo cáo này tổng hợp phần cài đặt ba thuật toán sắp xếp trong tệp `src/sorts.cpp` (gồm các hàm `selectionSort`, `insertionSort`, `quickSort`, cùng các hàm phụ trợ `SWAP`, `partition` và `quickSortH`), kèm phần phân tích cơ chế hoạt động và hiệu năng trên bốn kịch bản dữ liệu thử nghiệm: ngẫu nhiên (`random`), gần như đã sắp xếp (`nearly_sorted`), đảo ngược hoàn toàn (`reverse`) và trùng lặp 98% (`few_unique`) ở cả hai quy mô kích thước nhỏ (N = 0–500) và lớn (N = 10,000–20,000).

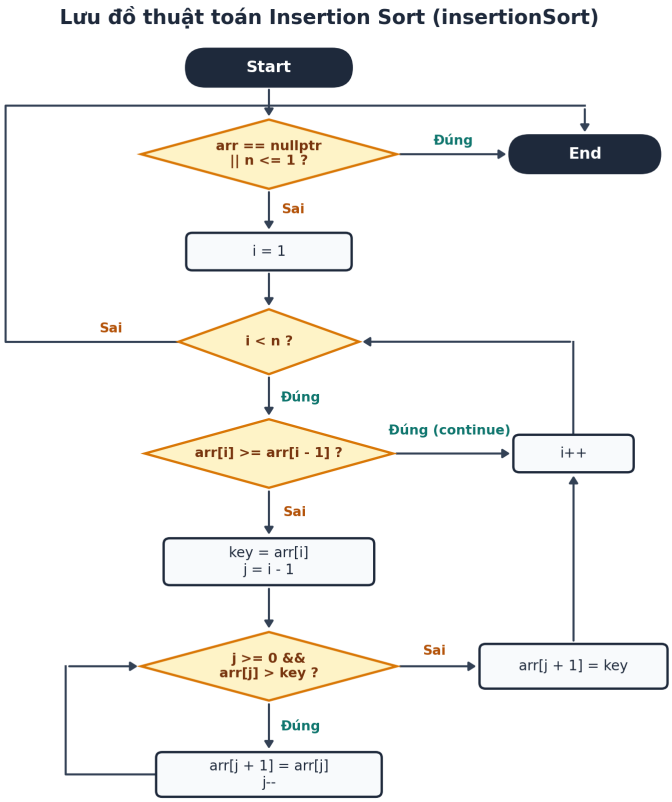
Selection Sort	**Insertion Sort**	**Quick Sort**
Hàm: selectionSort() Tìm minidx và SWAP cuối mỗi lượt Độ phức tạp cố định: $O(N^2)$	Hàm: insertionSort() Có nhánh bỏ qua arr[i] >= arr[i-1] Tốt nhất: O(N) | Xấu nhất: $O(N^2)$	Hàm: quickSort(), quickSortH(), partition() Phân hoạch Lomuto, chốt arr[high] Trung bình: O(N log N) | Xấu nhất: $O(N^2)$

1. Lưu đồ thuật toán (Flowchart)

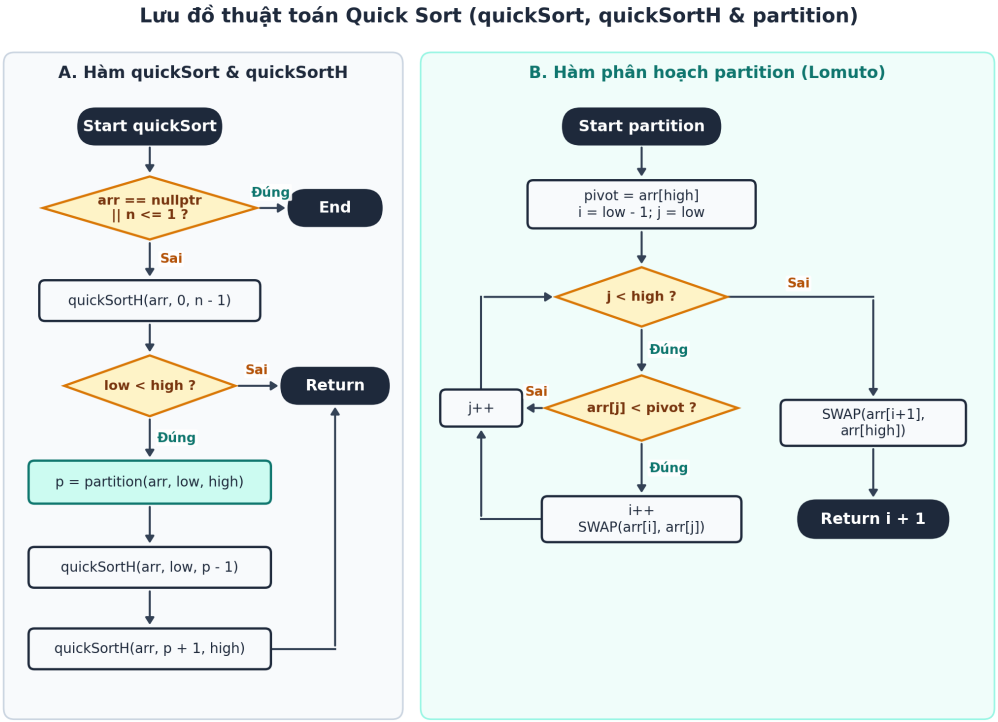
Ba lưu đồ dưới đây mô tả luồng điều khiển chính xác theo mã nguồn đã cài đặt, tuân thủ quy chuẩn ký hiệu lưu đồ: khối oval (Terminal) cho điểm bắt đầu/kết thúc, hình chữ nhật (Action/Process) cho phép gán và gọi hàm `SWAP`, hình thoi (Decision) cho các điều kiện rẽ nhánh với hai hướng **Đúng** / **Sai**, cùng mũi tên định hướng từ trên xuống dưới.



Hình 1.1: Lưu đồ thuật toán Selection Sort bám sát hàm selectionSort(arr, n).



Hình 1.2: Lưu đồ thuật toán Insertion Sort với điều kiện bỏ qua sớm arr[i] >= arr[i - 1].



Hình 1.3: Lưu đồ Quick Sort gồm hàm đệ quy quickSortH và phân hoạch Lomuto (partition).

2. Selection Sort qua 4 kịch bản dữ liệu: Thời gian chạy có thay đổi?

Khi quan sát đường biểu diễn của Selection Sort trên cả bốn bài đo (`random`, `nearly_sorted`, `reverse`, `few_unique`), thời gian thực thi gần như **không thay đổi đáng kể** giữa các bộ dữ liệu có cùng kích thước N.

Nguyên nhân nằm trực tiếp ở cấu trúc hai vòng lặp lồng nhau trong hàm `selectionSort`. Với mỗi chỉ số `i` từ `0` đến `n - 2`, vòng lặp trong luôn cho `j` quét trọn vẹn phần mảng chưa sắp xếp từ `i + 1` đến `n - 1` để tìm `minidx`. Thuật toán không có cơ chế dừng sớm (early exit) khi mảng đã có thứ tự, nên tổng số phép so sánh luôn cố định ở mức:

> **Số phép so sánh cố định:** (n - 1) + (n - 2) + ... + 1 = **n(n - 1) / 2** Độ phức tạp **O(N^2)** ở cả ba trường hợp Best, Average và Worst Case.
> *Ghi chú cài đặt:* Câu lệnh `if (minidx != i) SWAP(...)` giúp tránh hoán đổi thừa khi phần tử đã ở đúng vị trí (giảm số lần gọi `SWAP` trên mảng đã xếp), nhưng chi phí quét vòng lặp trong O(N^2) vẫn chiếm ưu thế tuyệt đối nên tổng thời gian hầu như giữ nguyên.

3. Khi nào Insertion Sort vươn lên vị trí dẫn đầu?

Insertion Sort đạt hiệu năng dẫn đầu trong hai bối cảnh rõ rệt: khi xử lý **dữ liệu gần như đã sắp xếp** (`nearly_sorted`), khi mảng có **nhiều phần tử trùng lặp liên tiếp** (`few_unique`), và trên các **mảng kích thước nhỏ (N <= 100)**.

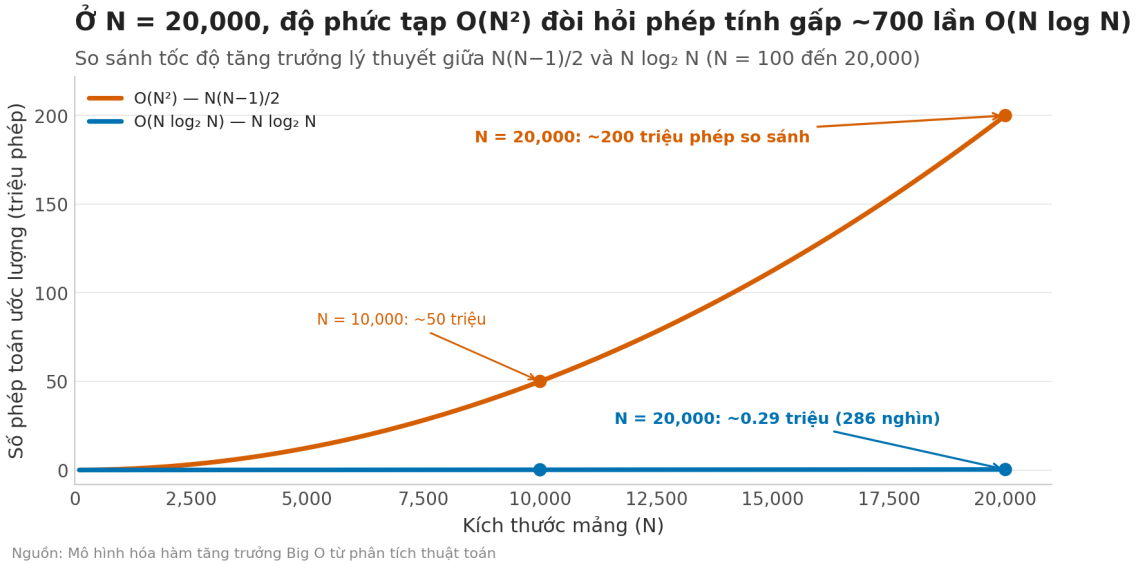
Cơ chế tạo ra ưu thế này xuất phát từ cấu trúc vòng lặp trong hàm `insertionSort`:

- **Nhánh bỏ qua trực tiếp (`if (arr[i] >= arr[i - 1]) continue;`):** Khi mảng gần như đã xếp tăng dần hoặc chứa nhiều giá trị bằng nhau kề nhau, điều kiện lớn hơn hoặc bằng đúng tại phần lớn vị trí `i`. Chương trình nhảy thẳng sang phần tử kế tiếp mà không cần khởi tạo `key` hay vào vòng lặp `while`.
- **Điều kiện dịch chuyển chặt (`while (j >= 0 && arr[j] > key)`):** Phép so sánh nghiêm ngặt `>` đảm bảo thuật toán giữ tính ổn định (stable sort) và không bao giờ dịch chuyển các phần tử có giá trị bằng `key`. Khi dữ liệu gần như có thứ tự, số lần dịch chuyển và gán giảm về gần 0, kéo thời gian chạy thực tế xuống tiệm cận **O(N)**.

4. Lý do Quick Sort bỏ xa hai thuật toán còn lại ở mảng lớn (N = 10,000 20,000)

Trên dữ liệu ngẫu nhiên phân bố đều kích thước lớn, tốc độ tăng trưởng độ phức tạp Big O quyết định toàn bộ khoảng cách hiệu năng. Theo phân tích tiệm cận, ký hiệu Big O phản ánh giới hạn tăng trưởng khi N tiến ra lớn sau khi bỏ qua hằng số và các hạng tử bậc thấp. Trên mảng ngẫu nhiên, phần tử chốt `pivot = arr[high]` thường chia đoạn mảng thành hai nửa tương đối cân bằng, tạo độ sâu cây đệ quy xấp xỉ $\log_2(N)$ tầng với chi phí O(N) mỗi tầng, đưa độ phức tạp trung bình về **O(N log N)**.

Ngược lại, cả Selection Sort và Insertion Sort đều giữ bậc tăng trưởng bình phương **O(N^2)** trên dữ liệu ngẫu nhiên. Khi kích thước N tăng gấp đôi từ 10,000 lên 20,000, khối lượng tính toán của O(N^2) tăng gấp 4 lần (từ 50 triệu lên 200 triệu phép so sánh), trong khi N $\log_2(N)$ chỉ tăng khoảng 2.15 lần (từ ~133 nghìn lên ~286 nghìn phép tính). Ở N = 20,000, chênh lệch khối lượng phép toán lên tới gần 700 lần, giải thích vì sao đường Quick Sort nằm sát trục hoành và bỏ xa hai đường còn lại trên biểu đồ lớn.



Hình 4.1: Mô hình tăng trưởng số phép toán giữa độ phức tạp $O(N^2)$ và O(N log N) từ N = 100 đến 20,000.

5. Vì sao trên mảng nhỏ (N <= 100), Quick Sort chậm hơn Insertion Sort?

Mặc dù có độ phức tạp tiệm cận tốt hơn ở N lớn, Quick Sort lại chịu bất lợi rõ rệt khi N <= 100 do chi phí vận hành đệ quy (recursion call stack overhead). Mỗi lần hàm `quickSortH` tự gọi lại chính nó:

- **Cấp phát khung ngăn xếp (Call Stack Frame):** Hệ thống phải đẩy một bản ghi mới lên ngăn xếp gọi hàm theo cơ chế LIFO, tạo bản sao riêng cho các biến cục bộ (`low`, `high`, `p`) và lưu địa chỉ trả về cho từng lời gọi.
- **Chi phí gọi hàm lồng nhau:** Với mỗi đoạn mảng có ít nhất 2 phần tử, `quickSortH` tiếp tục gọi `partition` và các phép hoán đổi qua `SWAP`, sau đó thu hồi bộ nhớ stack khi đệ quy lùi về điều kiện dừng (`low < high` sai).

Khi N <= 100, số lượng phép so sánh thực chất rất ít, khiến chi phí quản lý call stack và gọi hàm đệ quy lớn hơn chính công việc sắp xếp. Trong khi đó, `insertionSort` chạy hoàn toàn bằng vòng lặp phẳng, tại chỗ (in-place với bộ nhớ phụ O(1)), không tốn stack frame nên thực thi nhanh hơn ở quy mô mảng nhỏ.

6. Bảng so sánh tổng kết 3 thuật toán sắp xếp

Bảng dưới đây tổng hợp độ phức tạp thời gian theo ba kịch bản chuẩn (Best, Average, Worst Case), bộ nhớ phụ, tính ổn định và phân bố dữ liệu tối ưu/tồi tệ nhất ứng với chính cách cài đặt trong mã nguồn.

Thuật toán	Best Case	Average Case	Worst Case	Dữ liệu tối ưu nhất	Dữ liệu tồi tệ nhất
Selection Sort (O(1) phụ, Unstable)	$O(N^2)$	$O(N^2)$	$O(N^2)$	Mảng đã sắp xếp (số lần SWAP = 0, nhưng so sánh vẫn là $O(N^2)$)	Đảo ngược / ngẫu nhiên (số lần SWAP nhiều hơn; thời gian tổng thể gần như nhau)
Insertion Sort (O(1) phụ, Stable)	O(N)	$O(N^2)$	$O(N^2)$	Gần như đã sắp xếp (nearly_sorted) và trùng lặp nhiều (few_unique)	Đảo ngược hoàn toàn (reverse): mỗi phần tử key phải dịch chuyển tới đầu mảng

Thuật toán	Best Case	Average Case	Worst Case	Dữ liệu tối ưu nhất	Dữ liệu tồi tệ nhất
Quick Sort (Lomuto, chốt cuối; O(log N)–O(N) stack)	O(N log N)	O(N log N)	$O(N^2)$	Phân bố đều ngẫu nhiên (random): chốt chia mảng thành hai nửa cân đối	Đã sắp xếp, đảo ngược hoặc trùng lặp 98% (few_unique): chốt arr[high] gây chia lệch 0 và n - 1

7. Bonus: Vì sao phiên bản QUICK (Ref) không bị suy biến?

Trong cách cài đặt cơ bản của sinh viên, hàm `partition` luôn cố định chọn phần tử cuối `arr[high]` làm chốt và chia làm 2 vùng (`< pivot` và `>= pivot`). Do đó, khi gặp mảng đã xếp, đảo ngược hoặc có 98% phần tử bằng nhau (`few_unique`), mỗi lần phân hoạch chỉ tách được đúng 1 phần tử, đẩy độ sâu đệ quy lên N tầng và làm thuật toán suy biến về $O(N^2)$. Ngược lại, đường nét đứt `QUICK (Ref)` giữ được tốc độ ổn định trên mọi kịch bản nhờ xử lý triệt để ba điểm nghẽn này:

- **Chiến lược chọn chốt thông minh (Randomized Pivot hoặc Median-of-Three):** Thay vì lấy cứng phần tử cuối, việc chọn chốt ngẫu nhiên hoặc lấy trung vị của 3 phần tử (đầu, giữa, cuối) giúp chốt đại diện gần giá trị trung vị, loại bỏ tình trạng chia lệch cực đoan trên dữ liệu đã sắp xếp hoặc đảo ngược.
- **Xử lý tốt khóa trùng lặp (Phân hoạch 3 vùng / Hoare Partition):** Với dữ liệu có tới 98% phần tử giống nhau (`few_unique`), phân hoạch 3 vùng (nhỏ hơn, bằng, lớn hơn chốt) gom toàn bộ các phần tử bằng chốt vào giữa và loại chúng khỏi các lần gọi đệ quy tiếp theo, giúp tránh suy biến $O(N^2)$.
- **Thuật toán lai (Hybrid Sorting như IntroSort):** Kết hợp Quick Sort cho mảng lớn và tự động chuyển sang Insertion Sort khi kích thước đoạn con đủ nhỏ, vừa triệt tiêu chi phí call stack đệ quy ở mảng nhỏ, vừa giữ được ngưỡng O(N log N) ổn định.

> **Lưu ý về phạm vi dữ liệu:** Các biểu đồ định lượng cụ thể (tính theo đơn vị microgiây µs) được sinh trực tiếp từ công cụ đo của dự án trong thư mục `graphs/` khi chạy `make run`. Báo cáo này đối chiếu cấu trúc mã nguồn trong `src/sorts.cpp` với mô hình tăng trưởng độ phức tạp lý thuyết để giải thích hành vi của từng đường đồ thị trên cả hai dải kích thước small (0–500) và large (10,000–20,000).